

Model checking for Self-Replicating Robotic Systems

Initial Research

Master's Thesis

Dmitry Babaytsev (124065)

1. Referee: Prof. Dr. Jan Oliver Ringert
2. Referee: Prof. Dr. Unknown Yet

Submission date: June 31, 2025



Declaration

Unless otherwise indicated in the text or references, this thesis is entirely the product of my own scholarly work.

Weimar, June 31, 2025

Dmitry Babaytsev (124065)

Abstract

A short summary.

Contents

1	Introduction	1
2	Background	2
2.1	Formal verification and model checking	2
2.1.1	SAT solvers	2
2.1.2	SMT	4
2.1.3	Model checking	5
2.1.4	Classification of Model checking methods	5
2.1.5	Comparison of Model checking methods	5
2.1.6	Application of Model checking	6
2.2	Self-replicating robotic systems	6
3	Research Questions	7
3.1	Approach	7
3.2	Analysis of results	8
3.3	Evaluation	8
	Bibliography	9
A	My First Appendix	10

1 Introduction

Let us get started by citing [Bec+13]! So what did Beck et al. do in 2013? Maybe it is answered in chapter 1 on page 1?

You can look it up on https://en.wikipedia.org/wiki/Pythagorean_theorem.

2 Background

2.1 Formal verification and model checking

Formal Methods refers to mathematically rigorous techniques and tools for the specification, design and verification of software and hardware systems.

Expanding this definition Formal methods can be used for formal description of the system (specification) on some level of abstraction, program synthesis (design) according to specification and verification of a system to prove that model of system under consideration satisfies some properties.

These methods and techniques include:

1. SAT solving - operate on propositional logic
2. SMT solving (and tools like Z3) extend this to first-order logic, integrating theories such as arithmetic or arrays.
3. Relational model finding (e.g., Alloy)
4. Model checking (e.g., NuSMV) exhaustively explore state-space against CTL/LTL specifications.
5. Interactive theorem provers (e.g., Coq/Rocq, Isabelle/HOL)

2.1.1 SAT solvers

Propositional logic - consider propositions and relations between them and constructions of argument based on them.

Propositions - facts or statements about the world that can be true or false. Also defined as atomic propositions or Atoms. Some propositions can be described in natural language as atomic or indecomposable declarative sentences)

Atomic propositions can be connected to each other with logical connectives and compose propositional formulas. The most common connective relations are:

2 Background

- Negation
- Conjunction
- Disjunction
- Implication

Based on these we can construct a system of reasoning: from some set of formulas we can infer (prove) another formula that can be true or false:

$$\phi_1, \phi_2, \phi_3, \dots, \phi_n \vdash \psi$$

Such formulas called sequents, where on the left side there is a set of formulas called premises and on the right side - a formula called conclusion

Another relation between logical formulas is semantic entailment:

$$\phi_1, \phi_2, \phi_3, \dots, \phi_n \models \psi$$

it means that if propositional logic formulas $\phi_1.. \phi_n$ evaluate to True, then formula ψ also evaluates to True.

The formula ϕ is valid if for all valuations of atomic propositions in formula it evaluates to True:

$$\models \phi$$

SAT solver is a program which aims to solve the Boolean satisfiability problem (SAT), where the Boolean satisfiability problem sometimes called propositional satisfiability asks whether there exists an interpretation that satisfies a given Boolean formula.

Given formula ϕ is **satisfiable** if it has a valuation in which it evaluates to True.:

In other words, it asks whether the formula's variables can be consistently replaced by the values TRUE or FALSE to make the formula evaluate to TRUE. If this is the case, the formula is called satisfiable, else unsatisfiable.[13] SAT problem is NP-complete, as a result, only algorithms with exponential worst-case complexity are known. In spite of this, efficient and scalable algorithms for SAT were developed during the 2000s, which have contributed to dramatic advances in the ability to automatically solve problem instances involving tens of thousands of variables and millions of constraints.

SAT solvers usually begin by converting a problem to conjunctive normal form (CNF). **Conjunctive normal form (CNF)** - as a conjunction of clauses, where each clause is a disjunction of literals. Literal is an atomic proposition or a negated atomic proposition. Conjunctive normal form allows easy check of validity of a formula which otherwise take times exponential to number of atoms.

2 Background

2.1.2 SMT

Propositional logic is limited to express relations between logical components other than equivalent to natural language: *not, and, or, if..then* and declarative sentences that can only be True or False.

In order to be able to operate with finer logical structures we need to enrich the language. The necessity to solve this problem lead to design of Predicate logic or first-order logic.

In First-order logic we introduce:

- Variables that represent individual objects.
- Functions $\mathcal{F}$ that refer to objects.
- Predicates $\mathcal{P}$ - symbols that semantically represents some property or relation.
- Quantifiers: universal $\forall$ and existential $\exists$.

The Backus Naur form (BNF) for predicate logic:

$$\phi ::= P(t_1, t_2, \dots, t_n) \mid (\neg\phi) \mid (\phi \vee \phi) \mid (\phi \wedge \phi) \mid (\phi \rightarrow \phi) \mid (\forall x\phi) \mid (\exists x\phi)$$

Where,

t - called Terms and can be variable, constants or functions on terms $t ::= x \mid c \mid f(t, \dots, t)$

Atomic formulas are predicates applied to terms: $P(t_1, t_2, \dots, t_n)$

Variables can be free (supplied from outside the formula) or bounded by some quantifier.

Unlike propositional logic, in predicate logic we cannot just evaluate formulas based on given valuation by assigning True or False to the variables. We require a model of all function and predicate symbols involved. A model M for a first-order logic contains:

- Domain A is a non-empty set of individuals, mapping each variable to a value in A
- Set of functions where n-ary function is interpreted as a concrete function $f^m : A^n \rightarrow A$.
- Set of predicates n-ary predicate is interpreted as a relation $P^m \subseteq A^n$

Truth in a model is defined recursively:

- $M \models P(t_1, \dots, t_n)$ iff $(t_1, \dots, t_n) \in P^m$

2 Background

- $M \models \forall x \phi$ iff for every $a \in A$, $M \models [x \rightarrow a]\phi$
- $M \models \exists x \phi$ iff for some $a \in A$, $M \models [x \rightarrow a]\phi$

A sentence has no free variables, so its truth value depends solely on M .

We write $\Gamma \models \psi$ (semantic entailment) when every model that makes all formulas in Γ true also makes ψ true.

Soundness and completeness ensure $\Gamma \vdash \psi$ (provability) exactly when $\Gamma \models \psi$, but the two perspectives complement each other: proofs are constructive, while counter-models establish non-entailment.

2.1.3 Model checking

Model checking is a core approach in Formal verification that involves an algorithmic search of the state space of a system model to check if a given property (specification) holds, where:

Model - is an abstract representation of the system (often a state-transition graph or a program) capturing its possible states and transitions. The model can be described in a modeling language (like Promela for SPIN, or as a transition system in a tool like NuSMV).

Specification - is a formal property that the model should satisfy, commonly expressed in a temporal logic such as LTL or CTL. For example, an LTL specification might state that “whenever request happens, eventually response occurs” or a CTL property might require that “in all execution paths, a certain error state is never reached.”

2.1.4 Classification of Model checking methods

2.1.5 Comparison of Model checking methods

State explosion - the exponential growth of the state space with system parameters (like number of components or data bits). Combating state explosion is a central theme in model checking research

2.1.6 Application of Model checking

Hardware model checking

Software model checking

Reactive systems model checking

2.2 Self-replicating robotic systems

Reactive system: a system that maintains an ongoing interaction with its environment, responding to inputs with outputs continually (examples include operating systems, web servers, control systems). Reactive systems are typically non-terminating and concurrency-intensive.

3 Research Questions

This paper will address the following research questions:

1. How can automated model checking and other formal-verification techniques be used for verifying liveness (replication reliability) and safety properties in self-replicating robotic systems(SRRS), and what domain-specific adaptations are required for practical deployment?
2. What systematic methodology can translate the behaviours and structural constraints of self-replicating robots into expressive temporal-logic specifications?
3. What fundamental limitations—computational, semantic, or modelling-related—do current face when applied to highly dynamic, self-replicating robotic collectives, and how might these limitations be mitigated?

3.1 Approach

Experimental platform

As a subject of a research a simple robot will be taken, with two grippers and an elbow joint that can manipulate blocks (cuboctahedral voxels) one at a time. An experimental platform will be realised as a simulation in Gazebo environment with control from ROS2 nodes. The blocks and environment geometry robot joints and sensors and connections will be defined in Xacro/URDF files.

Generation of a model and specifications for formal-verification

A custom program has to be written in Python, that parses the Xacro files and ROS2 Nodes controller to generate a NuSMV model:

- a Boolean occupancy matrix representing the neighbourhood on a field relevant to the current replication.
- replication program-counter plus local flags (gripper-occupied, block -held, etc.).

3 Research Questions

The liveness and safety properties has to be defined independent from the simulation, in a form close to the following:

- Liveness. $G(start \rightarrow F(Robot_1 == ready \wedge Robot_2 == ready))$ – once replication starts, eventually both robots are able to function independently.
- Collision-free safety. $G\neg collision$ (or $AG\neg collision$ for CTL).

3.2 Analysis of results

3.3 Evaluation

Bibliography

- [Bec+13] Stephan Beck et al. “Immersive Group-to-Group Telepresence”. In: *IEEE Trans. Vis. Comput. Graph.* 19.4 (2013), pp. 616–625. DOI: 10.1109/TVCG.2013.33 (cit. on p. 1).
- [13] “Boolean satisfiability problem”. In: *IEEE Trans. Vis. Comput. Graph.* 19.4 (2013), pp. 616–625. DOI: 10.1109/TVCG.2013.33 (cit. on p. 3).

A My First Appendix

This was just missing.